

Task sheet 5 – Behavioral attractors and internal dynamics

Deadline: 17.12.2025

Objectives:

- ▶ implementing and solving a task based on behavioral attractors
- ▶ implementing a recurrent network and solving a complex task
- ▶ getting to know difficulties of reproducing results of other groups

1 Survival by autonomous recharging

We implement and solve the ‘navigate and recharge’ task (see also book p. 135). The robot is placed in an empty arena (no walls). At one corner there is a lamp and the floor at that corner is painted black within a quadrant. This black area represents the charging area. The robot has a simulated battery level $b \in [0, b_{\max}]$ that is immediately recharged ($b = b_{\max}$) once the robot enters the black area. We assume a fast linear discharge rate. The robot has three proximity sensors (to the front, 90° to the left, 90° to the right), two ambient light sensors (one at the front, one at the back, they are sensing a light intensity field similar to the Braitenberg vehicles in tutorial 1), one ground sensor (discrete, 0 for black, 1 for non-black), and a sensor that gives the battery level. **Implement an ANN with hidden layer.** You can choose any number of hidden neurons, preferentially between 2 and 5. For this task it might be advantageous to implement a **recurrent network**. To do so, you can provide each hidden neuron with its activation of the previous time step (as net input in addition to the regular inputs and multiplied with a dedicated weight).

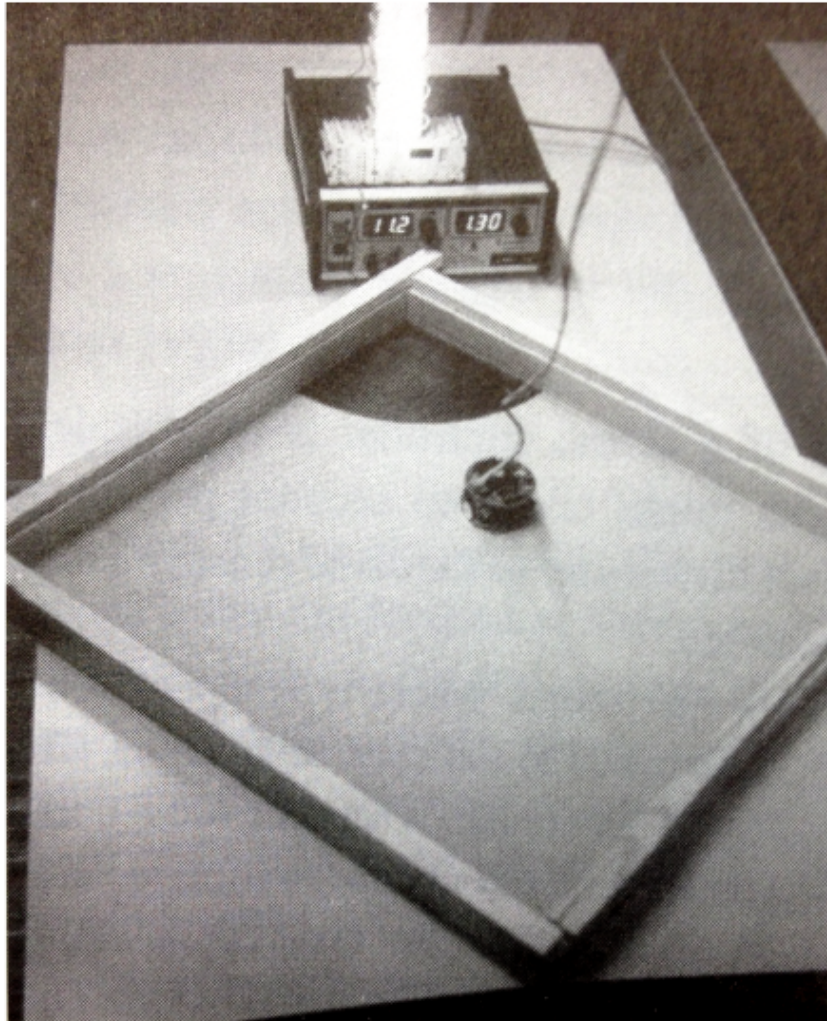
The fitness function is generally defined by:

$$\Phi = V(1 - i)$$

with $0 \leq V, i \leq 1$. V is the average rotation speed of both wheels. i is the average activation level of the sensors ($i \approx 0$ means no close-by objects, $i \approx 1$ means many close-by objects). However, Whenever the robot is positioned on the charging area it receives no fitness for that period.

Try to find good parameters for robot speed, discharging rate, size of the charging area, maximal battery level $b_{\max}$, arena dimensions, proximity sensor range, maximal evaluation time, etc. Remember that they need to relate to each other in an appropriate way to provoke the evolution of interesting behaviors (e.g., an excessively high robot speed combined with a very large maximum battery level makes the task trivial). You might want to base the fitness value on several independent evaluations with random initial positions. Consider also to add a little noise to the motor control values to avoid too simple behavioral patterns.

Prepare for long computations as the problem is hard and many evaluations will be needed. Try different settings until you are able to evolve a good behavior. Test your best controller after evolution has been done for different starting positions. Does it manage to reach the charging area from many different initial positions?



)

Your submission:

- ▶ please zip your submission in a single file named:
`evoRobo_sheet5_YOURLASTNAME1_YOURLASTNAME2.zip`
- ▶ a document with full names of group members, tasks completed or not completed
- ▶ accepted plot formats: png, jpg
- ▶ optional but preferable: a video explaining your code

For task 1 please turn in:

- ▶ all your code (C/C++ or Python)
- ▶ a plot of best and average fitness over generations
- ▶ a typical robot trajectory (plot/video) and a plot of battery level over time
- ▶ report the required changes you made (ANN setup, experiment parameters, etc.)